

자율 연구 보고서

지도 없이 라이다로 달리는 1:10 자율주행 레이싱카 제어기 개발

초파리 커넥툼 강화학습에서 40 Hz 비대칭 SAC, 실차 주행까지

아주대학교 기계공학과
ROBORACER 2026 출전팀

연구 기간 2026. 9. 18 ~ 2026. 10
2026년 10월

목차

요약

1. 연구 배경과 목표

1.1 출발점 · 1.2 연구 질문 · 1.3 스스로 정한 원칙 · 1.4 전체 구성

2. 1단계 — 초파리 커넥툼 SAC

2.1 구조 · 2.2 시뮬레이터와 보상 · 2.3 결과 · 2.4 이 결과로 말할 수 있는 것

3. 2단계 — mapless40: 40 Hz 비대칭 SAC

3.1 다시 설계한 이유 · 3.2 정책 구조 · 3.3 보상 · 3.4 시뮬레이터

3.5 실험 기록 v1 → v4 · 3.6 결과 · 3.7 학습 인프라 문제와 해결

4. 실차 이식과 실차 주행

4.1 실차 규약 맞추기 · 4.2 팀 시뮬레이터 검증 · 4.3 실차 주행 결과 · 4.4 시뮬과 실차의 차이

5. 연구 경과 — 시간순 기록

6. 스스로 찾아 고친 결함

7. 한계와 향후 과제

참고문헌

요약

우리 팀은 ROBORACER 2026(구 F1TENTH) 출전을 준비하며, 기존의 지도·위치추정 기반 주행 스택을 대체할 수 있는지 묻는 연구를 진행했다. **"지도와 위치추정 없이, 라이다와 IMU가 지금 보는 것만으로 조향과 속도를 직접 내는 신경망 제어기를 만들 수 있는가?"**

연구는 두 단계로 진행했다. 1단계는 초파리 hemibrain 커넥톰의 배선을 시간 메모리로 쓰는 SAC 정책이고, 2단계는 실차 조건을 세밀하게 넣어 다시 설계한 **mapless40**이다 — 40 Hz, 1125빔 라이다, 서보·구동 지연과 능동 제동이 없는 구동계를 시뮬레이터에 넣고, 레이싱 라인은 critic에게만 보여 주는 비대칭 SAC로 학습했다. 대회 코스와 비슷한 좁은 맵(ifac, 팀 Cartographer 맵)은 학습과 평가에 모두 썼다.

주요 결과

- **mapless40 (31.8만 스텝)** — 시뮬레이터의 팀 맵에서 출발 4곳 모두 3바퀴 무사고(9.63~9.65 s), ifac 맵에서 최고 10.9 s(pure pursuit 11.5 s, FTG 18.0 s보다 빠름). 다만 ifac 완주는 4번 중 1번이고, 팀 맵 기록은 트랙 바깥 빈 공간을 쓴 결과라 실제 트랙 폭 안의 기록은 아니다.
- **실차 주행** — 시뮬레이터에서만 학습한 정책을 실차에 올려 지도·위치추정 없이 라이다·IMU만으로 **주행하는 데까지는 성공했다**. 그러나 시뮬레이터처럼 안정적으로 돌지는 못했고, 완벽한 주행에는 아직 미치지 못한다(4.3절).
- **커넥톰 비교** — 헤딩을 맞춘 출발에서는 커넥톰만 완주했지만, 무작위 출발(둘 다 0/12)과 다른 맵 10개(둘 다 2/10)에서는 차이가 없었다. 커넥톰이 낫다고 결론 내릴 근거는 아직 부족하다.
- **찾아서 고친 결함** — 커넥톰 신호 방향 반전, 시뮬레이터가 횡가속을 사실상 제한하지 못하던 문제, 공개 레이싱 라인이 벽에 붙어 있던 문제, 이어 학습 직후 정책 붕괴 등 10건 이상을 찾아 고쳤다(6장).

1. 연구 배경과 목표

1.1 출발점

팀은 ROBORACER 2026에 출전한다. 1:10 차량이 2D 라이다로 좁은 실내 코스를 달리며 랩타임을 겨루고, 장애물과 상대 차를 피해야 한다. 팀의 기존 실차 스택은 **지도 작성 → 위치추정 → 미리 계산한 레이싱 라인 → 경로 추종(Stanley)** 구조다. 지도와 위치추정이 정확할 때는 강하지만, 그 둘이 흔들리면 전체가 흔들린다.

1.2 연구 질문

지도와 위치추정 없이, 라이다·IMU·바퀴속도만 보고 조향과 속도를 직접 내는 신경망이 이 차를 대회 코스에서 몰 수 있는가? 시뮬레이터에서만 학습한 정책이 실차에서도 달리는가?

1.3 스스로 정한 원칙

- **정책 입력에는 지도를 넣지 않는다.** 센터라인·레이싱 라인은 학습 중 보상과 평가(privileged 정보)에만 쓰고, 실차에 올라가는 actor는 센서 값만 본다.
- **실차 조건을 시뮬레이터에 넣는다.** 실측한 제원은 그대로, 모르는 값(서보·구동 지연, 코너 한계)은 실차보다 불리한 쪽으로 잡는다.
- **실차 스택과 같은 토픽·규약을 쓴다.** 기존 센서·제어 노드는 그대로 두고 경로 계획·추종만 정책으로 바꿔 끼운다.
- **측정한 뒤에 주장한다.** 기존 제어기(FTG, pure pursuit)를 같은 시뮬레이터에서 먼저 돌려 넘어야 할 기록을 정한다.

1.4 전체 구성

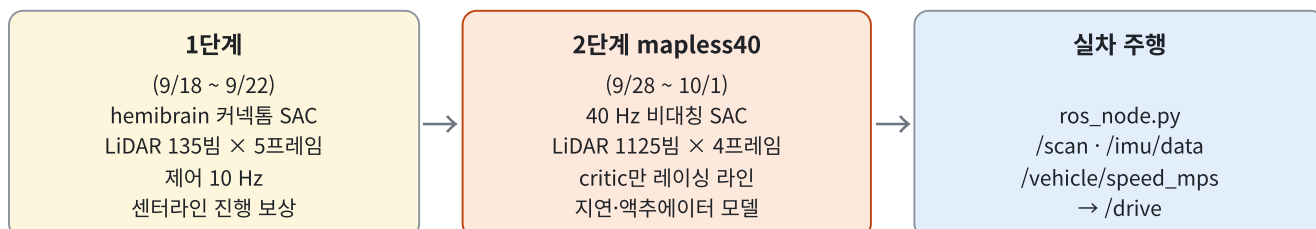


그림 1. 연구 전체 흐름. 팀 차량의 제원과 실차 스택의 토픽 규약을 시뮬레이터에 넣었고, 2단계 정책은 실차 스택의 센서·제어 노드 위에 Stanley 대신 얹혀 실차를 몰았다.

2. 1단계 — 초파리 커넥톰 SAC

첫 질문은 "초파리 뇌의 실제 배선을 LSTM 자리의 시간 메모리로 쓰면 주행 정책이 나아지는가"였다. 처음 PPO로 시작해 이틀날 연속 행동과 off-policy 학습에 유리한 SAC로 바꿨고, 같은 환경·같은 SAC에서 커넥톰 정책과 일반 MLP 정책을 비교했다.

2.1 구조

모듈	내용
관측 (680차원)	LiDAR 135빔 × 최근 5프레임 + yaw rate × 5. 지도·센터라인 없음
인코더	LiDAR 1D Conv → 48, yaw MLP → 16, 프레임별 $z \in \mathbb{R}^4$
ConnectomeRNN	neuPrint hemibrain v1.2.1에서 운동 감지(LC·LPLC 등)·중심복합체·하강뉴런(DN) 계열을 골라 최대 256개 뉴런. 연결 유무·부호는 고정하고 세기와 입력 가중치만 학습. 5프레임을 순서대로 넣어 갱신하고 DN 활성을 출력
skip · fuse · SAC	z 평균 → skip MLP, [skip; DN] → 256 → actor(조향, 속도) / critic

$$W_{\text{eff}} = A_{\text{signed}} \odot \text{scale} \odot M_{\text{topo}} \quad \# A_{\text{signed}}: \text{커넥톰 연결·부호 (고정)}$$

$$h \leftarrow h + (\Delta t / \tau) \cdot (-h + \tanh(h \cdot W_{\text{eff}} + W_{\text{in}} \cdot z_t)) \quad \# \text{leaky rate RNN}$$

hemibrain에는 T4/T5 운동 감지 뉴런이 없어 시각 투영뉴런(LC, LPLC 등)으로 대체했다. Jetson GPU 추론을 염두에 두고 GPU에서는 dense 행렬곱, CPU에서는 sparse 행렬곱을 쓴다.

2.2 시뮬레이터와 보상

- **물리**: 1차는 기구학 모델·2~3.5 m/s, 2차는 단일 트랙(ST) 동역학·2~7 m/s. 휠베이스 0.33 m, 조향 ± 0.3735 rad(팀 실측), 횡가속 한계 6 m/s².
- **타이밍**: 라이다 40 m / 40 Hz, 4프레임마다 행동 → 제어 약 10 Hz.
- **보상**: 센터라인 진행 거리 + 슬립각·횡가속 초과·조향 급변·횡오차 벌점, 충돌 -10, 랩 +30.
- **출발**: 센터라인 접선 방향으로 헤딩을 맞추고, 역주행 종료는 유예 시간 후 연속 프레임일 때만 판정해 가짜 역주행 종료를 줄였다.

2.3 결과

기구학 모델(2~3.5 m/s), **Spielberg**. plain MLP(15만 스텝)는 101.7 s, 커넥톰(약 8만 스텝)은 98.4 s로 둘 다 완주했다. 커넥톰은 1만 스텝에서 이미 랩을 돌아 초반 상승이 더 가팔랐다.

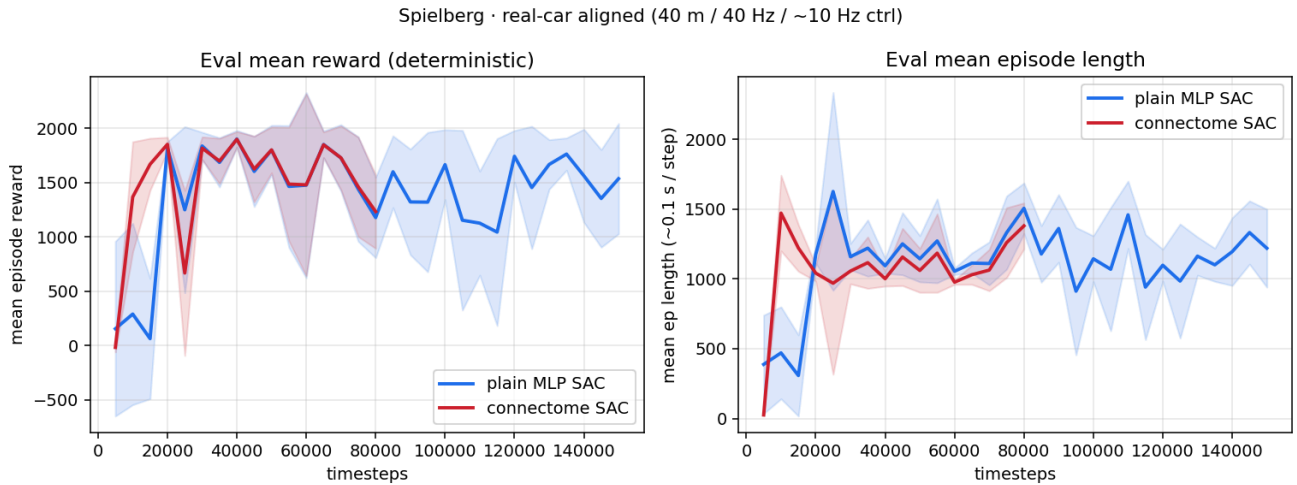


그림 2. 기구학 모델에서 두 정책의 평가 곡선. 둘 다 약 1만 스텝 이후 완주 구간(에피소드 길이 1,000~1,500)에 들어갔다.

ST 모델(2~7 m/s), Spielberg, 각 약 8만 스텝 최고 체크포인트:

측정	plain MLP	커넥툼
무작위 출발 12회 — 완주	0/12	0/12
무작위 출발 — 평균 / 최대 진행률	9% / 20%	16% / 78%
무작위 출발 — 충돌 / 역주행 종료	8 / 4	5 / 7
헤딩 정렬 출발 1회	93%에서 충돌	완주 (52.9 s)
정렬 출발 — 평균 속도 / 평균 횡가속	5.98 m/s / 3.61 m/s ²	6.40 m/s / 2.58 m/s ²

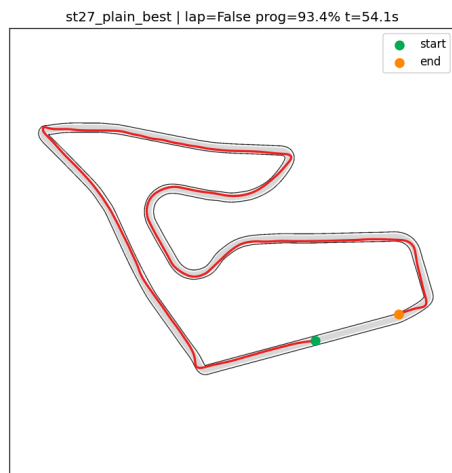


그림 3a. plain MLP 정렬 출발 레적 (93%에서 충돌).

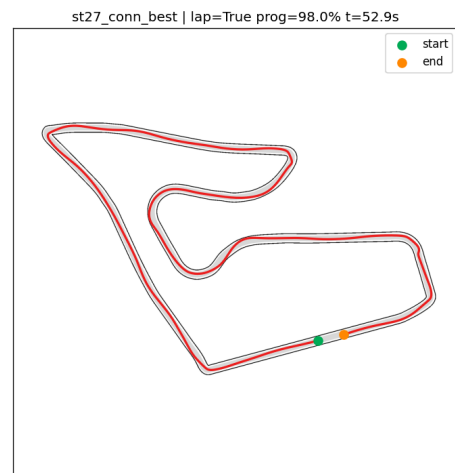


그림 3b. 커넥툼 정렬 출발 레적 (완주).

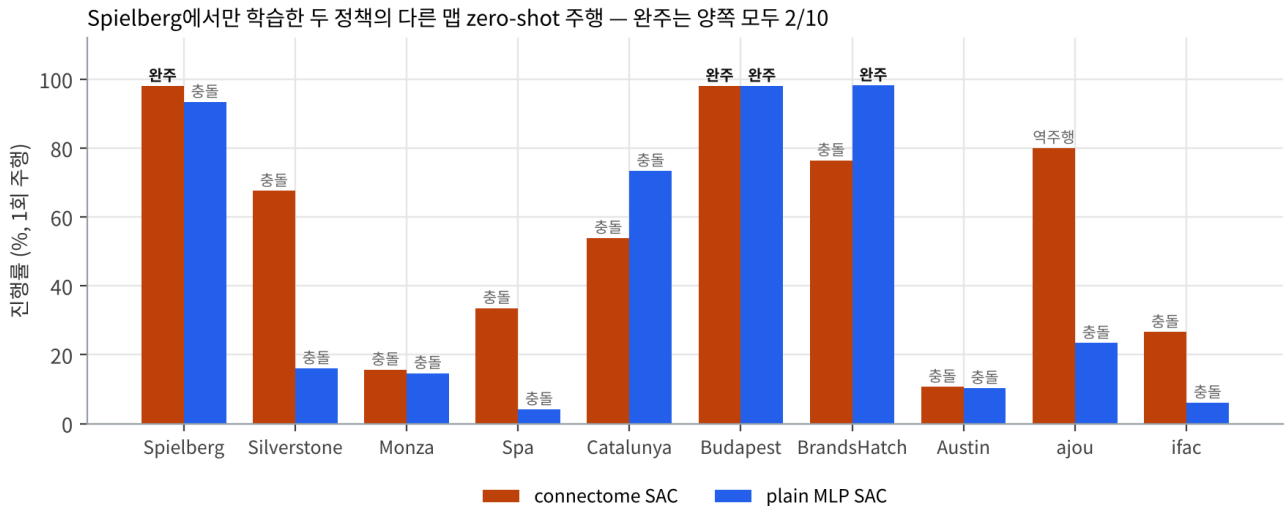


그림 4. Spielberg에서만 학습한 두 정책을 다른 맵 10개에 그대로 올린 결과(맵당 1회). 완주는 양쪽 모두 2/10이고, 대회 코스와 비슷한 좁은 맵(ajou, ifac)은 둘 다 실패했다.

2.4 이 결과로 말할 수 있는 것

커넥톰이 낫다고 결론 내리기에는 근거가 부족하다. 커넥톰이 앞선 완주 비교는 헤딩 정렬 출발 1회뿐이고, 무작위 출발(0 대 0)과 다른 맵(2 대 2)에서는 차이가 없었다. 학습 시드가 하나씩이라 우연과 구분할 수 없다.

- 이 단계 시뮬레이터에서 최대 횡가속이 $26\sim 27 \text{ m/s}^2$ (약 2.7 g)까지 나왔다. 1:10 차가 낼 수 없는 값이다. 2단계에서 원인을 찾았다 — 선형 타이어 ST 모델에서 마찰계수는 횡력 한계가 아니라 코너링 강성 배율이어서, 횡가속을 사실상 제한하지 못했다.
- 2단계 작업 중 GPU 경로의 커넥톰 신호가 거꾸로(후시냅스 → 전시냅스) 흐르던 버그를 찾아 고쳤다. 1단계 결과는 수정 전에 나왔으므로, GPU로 학습한 커넥톰 모델은 배선 방향이 뒤집힌 상태였을 수 있다.
- 좁은 대회형 맵을 전혀 못 도는 것이 가장 큰 문제였다. 이 두 가지가 2단계로 넘어간 직접적인 이유다. 커넥톰은 선택 모듈로 남겨 두었다.

3. 2단계 — mapless40: 40 Hz 비대칭 SAC

3.1 다시 설계한 이유

	1단계	mapless40	이유
제어 주기	10 Hz	40 Hz = 라이다 주기	실차는 스캔마다 명령을 낼 수 있음
라이다	135빔 × 5프레임	1125빔(0.24°) × 4프레임	LPX-T1 40 Hz 해상도에 맞춤
자기 상태	yaw rate	속도·yaw rate·직전 명령·스캔 간격 (17)	1틱 계산 지연을 상태에 포함
시간 결합	z 평균 (순서 소실)	4프레임 이어 붙임	순서 정보 보존
critic 입력	actor와 같음	actor 입력 + privileged 27	비대칭 actor-critic
보상 기준	센터라인 진행	레이싱 라인 진행 + 오차 + 목표 속도	빠른 라인을 가치로 전달
액추에이터	즉시 반응	서보·구동 지연, 능동 제동 없음	팀 control_node와 같게
할인율 γ	0.99	$0.99^{1/4} \approx 0.9975$	같은 시간 지평

3.2 정책 구조

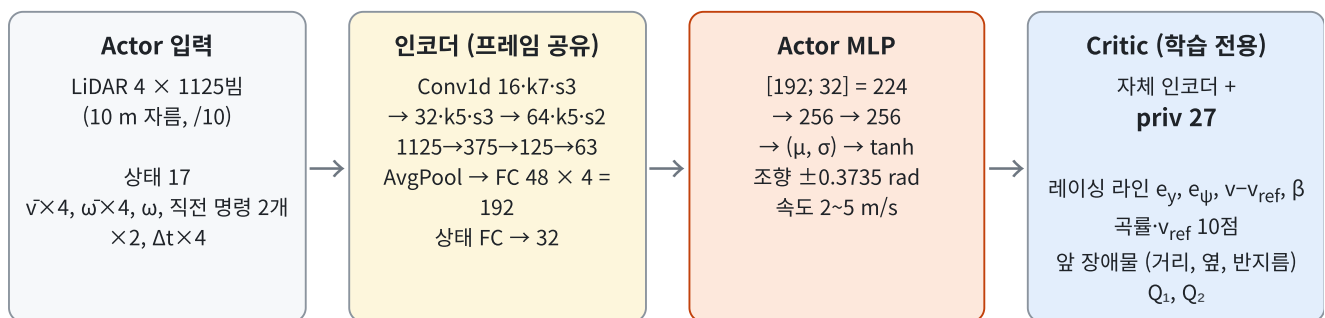


그림 5. mapless40 구조. critic만 레이싱 라인과 장애물 위치를 보고, 그 가치 신호가 기울기로 actor에 전달된다. 모방학습이 아니라 순수 강화 학습이다. 실차에는 actor만 내보낸다(TorchScript/ONNX, 또는 torch 없이 도는 numpy actor, 1.1 ms/틱).

다른 인코더도 구현했다: bev (IMU·속도로 4프레임을 현재 차 기준에 정렬한 150×150 격자 + 2D CNN), both (1D와 BEV 병렬), bev1 (최신 1장을 RViz base_link 화면과 같은 2×110×100 격자로). 2D CNN은 Colab 무료 T4에서 너무 느려 1D를 기본으로 두었다. 처음 1D CNN(32·64·64 채널, stride 2)은 업데이트 1회에 약 130 GFLOP라 초당 15스텝이었고, 채널과 stride를 줄여 연산을 1/6로 낮췄다.

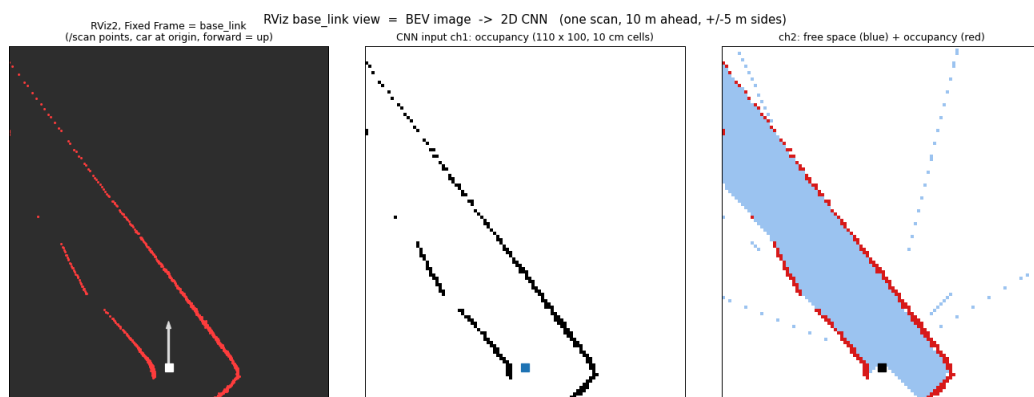


그림 6. 실차 RViz 화면(왼쪽)과 bev1 인코더가 보는 점유·빈공간 격자.

3.3 보상

$$r_n = 1.0 \cdot \Delta s - 0.5 \cdot e_y^2 \cdot \Delta t - 0.08 \cdot (v - v_{ref})^2 \cdot \Delta t \text{ (과속 } \times 3) - 0.1 \cdot |\delta_n - \delta_{n-1}| / \delta_{max} \\ - 2.0 \cdot \max(|\beta| - 0.05, 0) \cdot \Delta t - 1.0 \cdot [|v \cdot \omega| \text{가 한계의 } 80\% \text{ 초과분}] \cdot \Delta t - 2.0 \cdot [\text{벽 여유 } 0.25 \text{ m 부족분}] \cdot \Delta t \\ \text{충돌 } -20 \text{ (종료)} \cdot \text{랩 } +20 \cdot 1 \text{ s 연속 후진 시 종료}$$

시간 비례 항에 Δt 를 곱해 제어 주기를 바꿔도 보상 크기가 유지되게 했다. 레이싱 라인 준비 과정에서 두 가지를 고쳤다.

- 공개 레이싱 라인(f1tenth_racetracks)은 점 차량 기준이라 벽에서 0.1~0.3 m까지 붙는다. 차체로 따라가면 충돌하므로 거리장 기울기로 **벽에서 0.45 m 이상 밀어냈다**.
- 센터라인만 있는 맵(ifac)은 센터라인 기준 이론 랩이 14.6 s로 실제 대회 기록(9~10 s대)보다 한참 느렸다. elastic band 로 **최소곡률 근사 라인**(46.1 m → 40.8 m)을 만들어 썼다. 팀 맵은 공개 CSV가 맵 좌표와 맞지 않아 섬 둘레에 직접 점을 찍어 라인을 만들었다.

3.4 시뮬레이터

공식 시뮬레이터 대신 실차 조건을 넣을 수 있도록 직접 구현했다. 25 ms 한 스텝이 약 3 ms로 실시간의 약 8배다.

부분	모델	값의 근거
차량	ST, 선형 타이어, RK4 5 ms, μ 1.05 $\pm$ 10%, 횡가속 상한에서 요레이트 포화	휠베이스·조향각 팀 실측, 나머지 f1tenth_gym 값
서보	지연 15 ms + 1차 지연 50 ms + 각속도 4 rad/s	추정 (보수적)
구동	지연 30 ms, P 제어, 가속 5 m/s ² , jerk 40, 능동 제동 없음 , 타력 감속 0.6 + 0.15·v	제동 없음은 control_node 기준, 수치 추정
라이다	1125빔 EDT sphere tracing(2.5 ms/스캔, 1 cm 정밀 계산 대비 오차 $\leq$ 3 cm), 노이즈 2 cm, 누락 0.5%, 장착 오프셋	LPX-T1 사양, 팀 TF
IMU·속도	100 Hz 자이로(노이즈 + 에피소드별 바이어스), 50 Hz 속도	실차 주기
타이밍	계산 지연 10 ms, 틱 흔들림 5%, 스캔 누락 1%	추정
장애물	에피소드 70%에 원통 1~3개 (r 0.12~0.30 m, 통과 폭 $\geq$ 0.8 m). critic은 앞 10 m 장애물 위치를 봄	콘·박스 크기

학습 전에 기준 제어기를 같은 시뮬레이터에서 돌렸다. 지도 없는 FTG는 ifac 18.0 s, 지도를 아는 pure pursuit는 ifac 11.5 s, 최소곡률 라인 이론값은 11.0 s였다(v3 조건).

3.5 실험 기록 v1 → v4

버전	설정	결과와 판단
v1 (9/28)	1D + BEV 결합 인코더	업데이트 시작 후 초당 5스텝(100만 스텝에 50시간). BEV 빈공간 계산을 scatter → gather로 바꿨지만 여전히 느려 1D로 전환
v2 (9/28~29)	경량 1D CNN(초당 15 → 35스텝). 학습 맵 ifac + 넓은 F1 트랙 3개 균등, 라이다 15 m, 목표 엔트로피 -2	10만 스텝: ifac 2/3이 1바퀴(11.3 s) 후 충돌, Budapest 3/3 무사고. 18만 스텝: 팀 맵 4/4 완주. 43만 스텝: 팀 맵 0/4, ifac 0/4 — 넓은 트랙 운전엔 맞춰지며 좁은 트랙을 잊음. 엔트로피 계수 0.004로 탐색이 꺼짐
v3 (9/30)	맵 비율 ifac 3 : 팀 맵 3 : F1 3개 각 1, 목표 엔트로피 -1, 벽 여유 벌점, 라이다 10 m	20만 스텝: 팀 맵 3/3, ifac 1/3, 최고 9.75 s. 31.8만 스텝: 3.6절 . 그러나 코너의 절반 이상을 시뮬 마찰 한계(6 m/s ² , 실측 아님)에 붙어 돌았음
v4 (9/30~10/1)	횡가속 상한 6 → 4.5, 한계 80% 초과 시 벌점, 라인 속도 a_{lat} 5 → 3.6. v3 체크포인트에서 이어 학습	실차에서 미끄러질 주행을 줄이기 위한 조치. 이론 랩 ifac 11.0 → 12.3 s, 팀 맵 10.0 → 11.0 s로 느려짐을 감수

20만 스텝 모델은 출발 위치와 상관없이 같은 곳에서 실패했다 — ifac 아래 직선의 88~91% 지점, 그리고 팀 맵 왼쪽 헤어핀에 4.5 m/s로 들어가 최대 조향으로도 못 도는 경우다. 브레이크 없이 타력으로만 감속하는 차라 미리 줄이는 판단이 필요한데, 이것이 부족했다.

3.6 결과 (v3, 31.8만 스텝, 시뮬레이터)

조건 (출발 4곳 × 3바퀴)	결과	비교
팀 맵, 장애물 없음	4/4 무사고, 9.63~9.65 s	라인 이론 10.0 s, FTG 12.7 s
팀 맵, 장애물 2개	2/4 완주 (최고 9.18 s), 1번 1.9바퀴	—
ifac, 장애물 없음	1/4 완주, 2번 1~2바퀴 후 충돌, 최고 10.9 s	이론 11.0 s, pure pursuit 11.5 s, FTG 18.0 s
ifac, 장애물 2개	2/4 완주 (10.88~11.3 s)	—
Budapest (학습 안 한 트랙)	2번 중 1번 60 s 무사고	v2는 3/3

평균 속도는 3.8~4.0 m/s(명령 범위 2~5 m/s)였다.

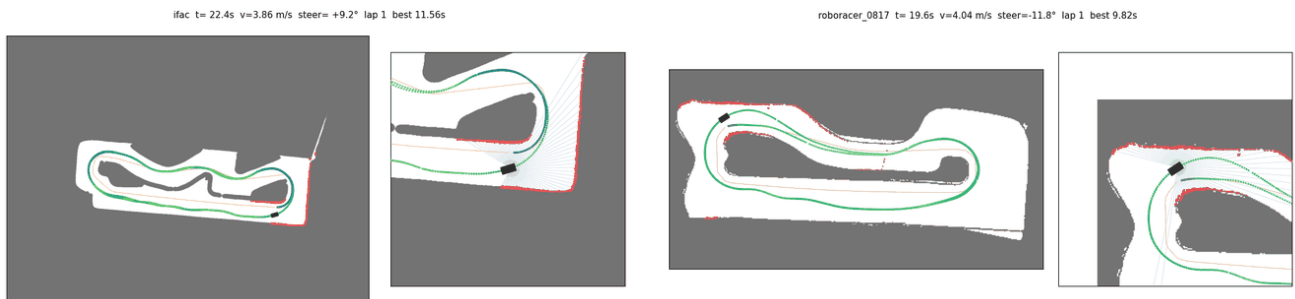


그림 7a. v3 · ifac 주행. 왼쪽은 트랙 전체와 궤적(속도 색), 오른쪽은 차 주변 10 m의 라이다 빔. 그림 7b. v3 · 팀 맵 주행. 섬 바깥의 빈 공간을 넓게 쓰는 구간이 있다.

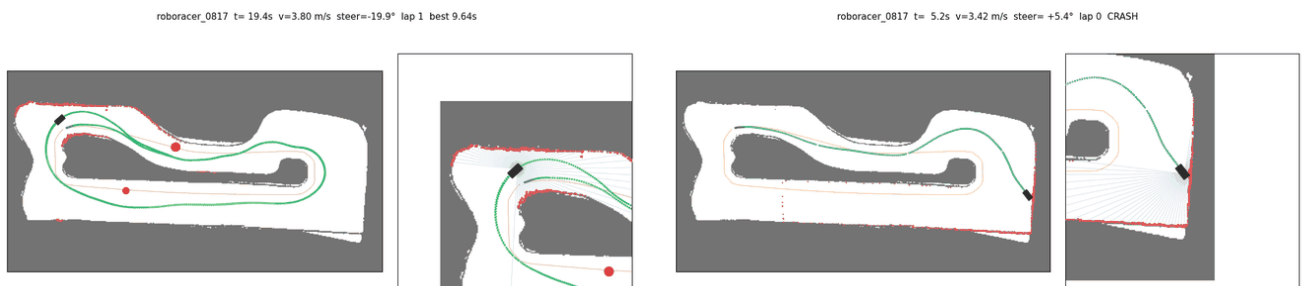


그림 7c. v3 20만 스텝 · 팀 맵 + 장애물 2개(빨강).

그림 7d. v2 43만 스텝 · 팀 맵 실패. 넓은 트랙에 치우친 정책.

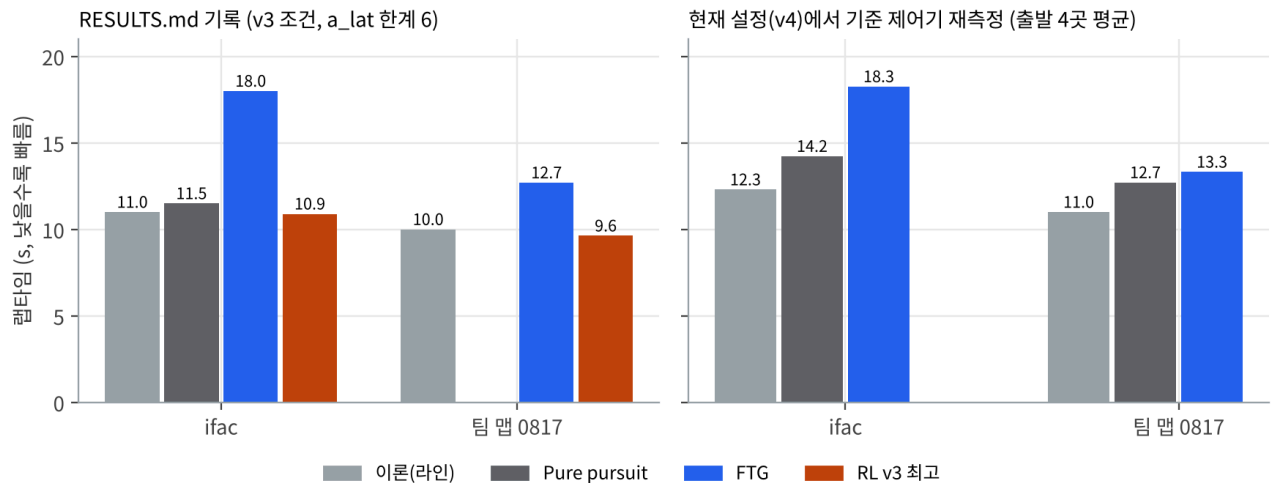


그림 8. 시뮬레이터 랩타임. 왼쪽은 v3 조건의 기록이고, 오른쪽은 횡가속 한계를 낮춘 현재 설정(v4)에서 기준 제어기를 다시 잰 값이다. 같은 pure pursuit가 ifac에서 11.5 s → 14.2 s로 느려지므로, v3 정책의 10.9 s는 왼쪽 조건에서만 비교할 수 있다.

이론 랩타임보다 빠른 기록은 그대로 믿으면 안 된다. 팀 맵 9.63 s는 라인 이론 10.0 s보다 빠르는데, Cartographer 맵에 실제 트랙 벽이 없는 방 바닥까지 빈칸으로 찍혀 있어 정책이 트랙보다 넓게 돈 결과다. ifac 10.9 s도 정책이 시뮬 횡가속 상한 6 m/s^2 (실측 아님)까지 쓴 결과다. 그래서 v4에서 한계를 낮췄다.

3.7 학습 인프라 문제와 해결

학습은 Colab 무료 T4 GPU로 했다. 세션이 수시로 끊기는 환경이라 인프라 문제가 결과만큼 시간을 잡아먹었다.

증상	원인	해결
Colab이 끊기면 결과가 없음	학습이 끝날 때만 저장	10분마다 마지막 모델, 10만 스텝마다 체크포인트
이어 학습 직후 정책 붕괴 (에피소드 길이 837 → 114)	SB3가 빈 리플레이 버퍼를 무작위 행동으로 채움	이어 학습 땀 불러온 정책으로 1만 스텝 수집
저장 중 pickle 오류	모델 인스턴스에 함수를 붙임	클래스 교체 방식으로 변경
재시작 시 BadZipFile	실패한 저장에 zip을 반쯤 덮어씀	임시 파일 저장 → 검증 → 교체, 깨진 zip 건너뛰
학습 속도를 알 수 없음	SB3 fps가 누적 평균	최근 1분 초당 스텝, 환경 대 업데이트 시간 비율, 남은 시간 출력
GPU를 쓴다고 생각했는데 느림	SB3가 조용히 CPU로 떨어짐	실제 장치를 로그에 출력
평가 기준이 바뀌었는데 예전 최고 모델이 유지됨	평가 조건이 기록되지 않음	최고 모델 기록에 평가 조건 태그, 바뀌면 이전 최고를 따로 보관

4. 실차 이식과 실차 주행

4.1 실차 규약 맞추기

정책을 실차에 올리기 위해 ROS2 노드(`ros_node.py`)를 만들었다. `/scan` 콜백 한 번이 제어 한 번이고, 지도와 인터넷은 필요 없다. 팀 스택의 센서·제어 노드(`slidar 40 Hz`, IMU 드라이버, 정적 TF, `control_node` 자율 모드)는 그대로 띄우고 Cartographer와 Stanley는 띄우지 않는다. 학습 때와 실차의 입력이 같도록 다음을 맞췄다.

항목	실차 스택의 실제 값	노드에서의 처리
<code>/scan</code>	360° 전체, 회전마다 점 수가 조금씩 다름	각도 기준으로 0.24° 격자 1125칸에 재배치(같은 칸은 최소값)
라이다 방향	TF yaw 기본 0, 실제 정면은 스캔 -177° 부근	시작 시 정지 스캔 40장으로 차체에 가린 구간이 뒤쪽인지 검사, 앞이면 명령을 내지 않고 보정값 출력
<code>/imu/data</code>	약 95 Hz, 시리얼로 몇 개씩 묶여 도착	수신 시각 대신 <code>header.stamp</code> 사용, ± 8 rad/s 클립
<code>/vehicle/speed_mps</code>	Float64, 50 Hz	Float64 구독 (처음엔 Float32로 받아 속도를 못 읽음)
<code>/drive</code>	조향 풀스케일 0.3735 rad, + = 좌	같은 규약, 첫 시험용 최고 속도 상한 파라미터

2초마다 스캔 주기, IMU·속도 수신 여부, 처리 지연을 로그로 남겨 지연이 25 ms 예산보다 충분히 작은지 확인한다. torch 없는 numpy actor는 한 틱에 1.1 ms가 걸린다.

4.2 팀 시뮬레이터 검증

실차 전에 팀이 쓰던 `f1tenth_gym_ros` 시뮬레이터에서 Stanley 자리에 정책을 꽂아 보았다. 어댑터 노드가 250 Hz 스캔을 40 Hz로 내리고 `odom`을 IMU·속도 토픽으로 바꿔 준다. gym은 서보·구동 지연이 없는 이상적 차량이라, 여기서 확인한 것은 토픽·타이밍 파이프라인과 트랙 모양에 대한 일반화까지다. 팀 맵의 미탐색 영역(gym은 이를 빈칸으로 읽음)은 벽으로 바뀌 넣었다.

4.3 실차 주행 결과

시뮬레이터에서만 학습한 정책을 Jetson에 올려 1:10 실차로 자율 주행 시험을 했다. 주행 중 지도와 위치추정은 쓰지 않았고, 라이다·IMU·바퀴속도만으로 조향과 속도를 냈다. 실차 데이터로 다시 학습하지는 않았다.

실차에서 주행은 됐다. 시뮬레이터에서만 배운 정책이 실차 센서 입력을 받아 트랙을 따라 달렸다. 그러나 시뮬레이터처럼 매끄럽고 안정적으로 돌지는 못했고, 완벽한 주행에는 아직 미치지 못한다.

실차 결과는 주행 가능 여부와 주행 상태에 대한 관찰이며, 랩타임 등 정량 기록은 이 보고서에 포함하지 않았다.

4.4 시뮬과 실차의 차이 — 원인 후보

완벽하지 않은 원인은 아직 실측으로 가려내지 못했다. 시뮬레이터를 만들 때 추정으로 남겨 둔 값과, 실차 스택에서 미해결로 기록된 문제가 우선 후보다.

후보	내용	확인 방법
서보·구동 응답	지연 15/30 ms, 1차 지연 50 ms, 각속도 4 rad/s는 모두 추정값	조향 step → IMU 요레이트, 속도 step → VESC 로그
타력 감속	브레이크 없는 차에서 코너 진입 속도를 정하는 핵심 값인데 추정값($0.6 + 0.15 \cdot v$)	목표 속도를 내리는 step 시험
코너 마찰 한계	v_3 는 6 m/s^2 , v_4 는 4.5 m/s^2 로 학습. 실제 바닥 마찰은 모름	원 선회 속도를 올리며 IMU 횡가속
IMU 축	IMU 축 방향이 하드웨어와 드라이버 사이에 맞는지 확인이 필요. 정책은 요레이트를 입력으로 받으므로 부호가 틀리면 회전을 반대로 읽음	차 앞을 왼쪽으로 돌릴 때 angular_velocity.z가 양수인지
라이다 방향·정렬	TF yaw와 드라이버 angle_offset이 각각 맞는지 정렬 확인 필요	시작 검사 로그, RViz 정렬
속도 상한	2~5 m/s로 학습한 정책을 첫 시험에서 낮은 상한으로 자르면 학습 때 겪지 않은 상태가 됨	상한별 주행 비교
트랙 차이	학습 맵(ifac, 팀 맵)과 시험 트랙의 폭·벽 재질·반사율 차이	시험 트랙 맵을 학습·평가에 추가

5. 연구 경과 — 시간순 기록

각 단계에서 무엇을 의도했고, 무엇을 발견했고, 어떻게 고쳤는지를 날짜 순으로 정리했다.

날짜	한 일	발견·판단
9/18	커넥툼 PPO로 mapless 학습 환경 시작	—
9/19	SAC로 전환, 커넥툼 시간 unroll 학습, GPU 경로, 40 m/40 Hz 라이다 + 10 Hz 제어	연속 행동에 SAC가 적합
9/20	Spielberg에서 plain vs 커넥툼 비교 (기구학)	둘 다 완주 (101.7 s / 98.4 s)
9/21	ST 동역학을 팀 제원에 맞추고 속도 2~7 m/s로 확대, 팀 Cartographer 맵 지원	출발 헤딩·역주행 판정 보강
9/22	ST 비교, 레이어 활성화 시각화	정렬 출발 1회만 커넥툼 완주
9/28 오전	다른 맵 10개 zero-shot, 40 Hz 비대칭 SAC 설계 수식 정리	둘 다 2/10 — 좁은 맵 전부 실패
9/28	mapless40 구현: 환경·정책·학습·ROS 노드, Colab 노트북, 2~5 m/s 간단 버전, 장애물, 팀 시뮬 연동, numpy actor	커넥툼 신호 방향 버그, ST 마찰 문제 발견. BEV 느낌 → 1D
9/29	이어 학습 결함 3건 수정, 주행 GIF 도구	재시작 직후 정책 붕괴 원인 해결
9/30	v3 설정, bev1 인코더, 실험 기록 정리, 저녁 v4	v2가 좁은 트랙을 잇음 → 맵 비율 조정. v3가 마찰 한계에 붙어 뚫 → 한계 낮춤
10/1	v3 모델을 v4 조건으로 이어 학습, 평가 조건 태그	—
이후	학습한 정책을 실차에 올려 자율 주행 시험	주행은 되나 완벽하지 않음 (4.3절)

6. 스스로 찾아 고친 결함

가장 자주 마주친 위험은 프로그램이 멈추는 오류가 아니라, 경고 없이 결과만 틀리게 나오는 결함이었다. 대부분 "숫자가 이상하다"거나 "설명이 안 된다"는 의심에서 출발해 직접 측정해서 찾았다.

#	결함	드러난 계기	고치지 않았다면
1	커넥툼 GPU 경로의 신호 방향이 반대	CPU 경로와 대조	배선이 뒤집힌 채 학습
2	ST 모델 마찰계수가 횡력 한계가 아니어서 횡가속 무제한(최대 27 m/s ²)	요레이트가 지구학의 68%	실차에서 불가능한 코너링 학습
3	공개 레이싱 라인이 벽에서 0.1~0.3 m	차체 기준 충돌 검사	보상 기준선이 벽 안
4	센터라인 기준 ifac 이론 랩 14.6 s (실제 9~10 s대)	대회 기록과 비교	목표 속도가 너무 낮음
5	속도 토픽 Float64를 Float32로 구독	실차 스택 코드 대조	실차 노드가 속도를 못 받음
6	IMU 수신 시각이 시리얼 배치로 뭉침	타임스탬프 분포	요레이트 구간 평균 왜곡
7	이어 학습 시 빈 버퍼를 무작위 행동으로 채움	재시작 직후 붕괴	재시작마다 학습 손실
8	저장 실패가 모델 파일을 반쯤 덮어씀	BadZipFile	Colab 끊김마다 결과 소실
9	v2가 넓은 트랙에 맞춰지며 좁은 트랙을 잇음	18만 → 43만 스텝 평가 하락	대회형 트랙에서 못 도는 모델 채택
10	v3가 코너 절반 이상을 시뮬 마찰 한계에 붙어 주행	횡가속 분포 확인	실차에서 미끄러질 정책 채택

7. 한계와 향후 과제

- **실차 주행이 아직 완벽하지 않다.** 시뮬레이터에서만 학습한 정책이 실차에서 달리는 것까지는 확인했지만 안정적인 완주에는 못 미친다. 4.4절의 후보를 하나씩 실측해 원인을 가려야 한다.
- **시뮬레이터 값 일부가 추정이다.** 서보·구동 지연, 타력 감속, 코너 마찰 한계, 계산 지연, 가속·jerk 다섯 묶음이 실측 전이다. 실차 step 시험으로 식별하고, 값을 바꾸면 재학습한다.
- **실험을 한 번씩만 돌렸다.** 모든 학습이 시드 하나, 평가는 출발 4곳 이하이다. v2 → v3 개선도 설정 효과와 학습 운을 구분할 수 없다. 시드 3개 이상과 더 많은 출발·섭동 조건으로 다시 재야 한다.
- **팀 맵 평가가 실제 트랙보다 넓다.** 맵에 실제 트랙 경계를 막아 넣고 다시 평가해야 한다.
- **커넥톰 효과는 판단하지 못했다.** 방향 버그 수정 후, 같은 학습량·여러 시드로, 같은 뉴런 수의 무작위 희소 배선 대조군과 비교해야 "초파리 배선" 효과와 "희소성" 효과를 나눌 수 있다.
- **v4 결과가 아직 없다.** 횡가속 한계를 낮춘 조건에서 다시 평가해야 한다.
- **상대 차와 추월은 아직 없다.** 시뮬레이터에는 정지 장애물만 있다. 동적 장애물(상대 차), 스캔 1회전 동안의 왜곡, 트랙 자동 생성으로 맵 편향 줄이기가 다음 후보다.
- **2D CNN 인코더 비교가 남았다.** BEV 계열 인코더는 구현했지만 계산 자원 부족으로 1D와 비교하지 못했다.

참고문헌

연구 중 설계와 구현에 사용한 알고리즘·데이터의 출처다.

강화학습

- [1] Haarnoja, T., Zhou, A., Abbeel, P., Levine, S. (2018). Soft Actor-Critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. ICML.
- [2] Pinto, L., Andrychowicz, M., Welinder, P., Zaremba, W., Abbeel, P. (2018). Asymmetric actor critic for image-based robot learning. RSS.
- [3] Raffin, A., et al. (2021). Stable-Baselines3: Reliable reinforcement learning implementations. JMLR, 22(268).

초파리 신경과학

- [4] Scheffer, L. K., et al. (2020). A connectome and analysis of the adult Drosophila central brain. eLife, 9, e57443. — 데이터: neuPrint hemibrain v1.2.1.

자율주행 레이싱 · 제어

- [5] O'Kelly, M., Zheng, H., Karthik, D., Mangharam, R. (2020). F1TENTH: An open-source evaluation environment for continuous control and reinforcement learning. PMLR 123.
- [6] F1TENTH. f1tenth_racetracks (공개 서킷 맵·센터라인·레이싱 라인 데이터).
- [7] Hoffmann, G. M., Tomlin, C. J., Montemerlo, M., Thrun, S. (2007). Autonomous automobile trajectory tracking for off-road driving: Controller design, experimental validation and racing. American Control Conference.
- [8] Sezer, V., Gokasan, M. (2012). A novel obstacle avoidance algorithm: "Follow the Gap Method". Robotics and Autonomous Systems, 60(9).
- [9] Coulter, R. C. (1992). Implementation of the pure pursuit path tracking algorithm. CMU-RI-TR-92-01.

위치추정

- [10] Hess, W., Kohler, D., Rapp, H., Andor, D. (2016). Real-time loop closure in 2D LIDAR SLAM. ICRA.